Are the reductions efficient?

An `efficient` keyword for Lean

Daira-Emma Hopwood
Zcash Open Development Lab
using Mythos 5.1

Lightning talk, Cambridge formal verification workshop
October 2026

The goal

A security reduction is a program. From an adversary $\mathcal{A}$ against the scheme it **constructs** an adversary $\mathcal{B}$ against the assumption (or lower-level scheme). In Lean, the reduction R is just a function, and composition is just function composition. Expressed in terms of advantage functions:

$$\mathrm{Adv}_{\mathcal{B}} \geq \mathrm{Adv}_{\mathcal{A}} - \varepsilon_R \qquad \text{and} \qquad \mathrm{time}_{\mathcal{B}} \leq k \cdot \mathrm{time}_{\mathcal{A}} + \mathrm{time}_R.$$

In the Ironwood formalization, `assert_computable R` pins the first half: `R` is a plain `def`, and every axiom that it reaches is declared (`+choice`, `+native`).

What about time_R? We *can* just eyeball the reduction and check it's not doing unbounded work. That is… probably fine?

Doing better than "probably fine"

`assert_computable` does not pin the time overhead. A `def` can be computable and exponential, and the theorem then does not say anything about concrete security.

Wanted: `assert_efficient R`, with the same discipline. Desired properties:

- It fails the build unless the cost of `R` can be derived, and gives a useful error where the derivation stopped.
- The cost should be concrete, derived from the definition itself, and symbolic in the adversary.
- It should handle higher-order functions and instance polymorphism, which are common in Lean.

Previous approaches

- **Isabelle,** `time_fun` (HOL-Library): T_f derived from f's own equations, one unit per call; T_{map} takes T_f as a parameter; `+` , `<` , `=` declared constant-time. Nipkow's book proves the bounds by hand, usually a short induction.
- **CSLib** `TimeM` : a cost monad with user-placed ticks. Cooperative: a `def` that works outside the API is not seen.
- **VCVio**: enforced query bounds; polynomial time as an abstract predicate, with no certificate constructed yet.
- **EasyCrypt** cost logic (Barbosa, Barthe, Grégoire, Koutsos, Strub 2021): a Hoare logic for the concrete cost of adversarial code, intrinsic cost plus calls to abstract modules. The closest in spirit, not in Lean.
- **Forster and Kunze** (ITP 2019): the user states the bound, a tactic proves it against an extracted term. The "just works" model for bounds.

Why it can be subtle

- **The adversary is a parameter.** Its cost is unknown, so the reduction's cost is a *function of the adversary's cost*. Higher order is the common case: adversaries, oracles, and interfaces are functions, or structures of functions.
- **Recursion needs a termination argument.** The cost function recurses as the function does. Lean finds the argument for structural recursion. For well-founded recursion the measure is effectively a loop variant, but it lives in a separate proof, not in the equations.
- **Polymorphism and type classes.** `+` on an abstract field has no fixed cost. An instance parameter is a bundle of functions whose cost is (usually) unstated.
- **The term is not the source.** `match` and `if` elaborate to matchers, `casesOn`, and `Decidable.rec`; there are casts, `let`-bound functions, induction hypotheses, and `opaque` barriers. The price has to be read off the *elaborated* term.
- **Costing should be non-cooperative.** A cost monad doesn't see any `def` outside it.

The solution: `assert_efficient f`

Derives `f.cost`, with `f`'s binders and returning `Nat`, from `f`'s definition, and then the same for every callee without one, down to the primitives:

- a call costs one plus the callee's cost at the actual arguments;
- `if` and `match` cost one, then the branch taken; recursion recurses as `f` does;
- constructors, literals, projections, and casts are free;
- a primitive needs a registered cost: arithmetic is one unit per operation, as in Isabelle's `time_fun_0`, with the same caveats.

The solution: `assert_efficient f`

```
def sumList : List Nat → Nat
  | [] => 0
  | x :: rest => x + sumList rest

assert_efficient sumList              -- derives sumList.cost
#eval sumList.cost [1, 2, 3]          -- 13
theorem sumList_cost_le (xs : List Nat) :
    sumList.cost xs ≤ 4 * xs.length + 1 := by
  induction xs with
  | nil => simp [sumList.cost]
  | cons x rest ih => simp [sumList.cost, hAddCost]; omega
```

The subtle cases, 1: higher-order functions

A function-typed parameter gets a **companion**, its cost function. A structure of functions gets one companion per field that is called, keyed by the parameter and the field:

```
def applyTwice (f : Nat → Nat) (n : Nat) : Nat := f (f n)
-- applyTwice.cost : (f : Nat → Nat) → Nat → (f.cost : Nat → Nat) → Nat
#eval applyTwice.cost (· + 1) 3 (fun _ => 1)        -- 4

structure Oracle where (query : Nat → Nat) (bound : Nat)
def twoQueries (o : Oracle) (x : Nat) : Nat := o.query x + o.query o.bound
#eval twoQueries.cost ⟨(· * 2), 5⟩ 3 (fun _ => 2)   -- 8
```

- A lambda argument's companion is its body's cost; a global function's is its cost function; `List.map` derives its own, with companions.
- An instance parameter is a structure of functions too: a method called on it gets a companion keyed by the instance and the method.

So the reduction's cost is *symbolic in the adversary's*: $\mathrm{time}_{\mathcal{B}} \le k \cdot \mathrm{time}_{\mathcal{A}} + \mathrm{time}_{R}$.

The subtle cases, 2: recursion

- **Structural recursion** is free: the cost function is structural over the same argument.
- **Well-founded recursion:** Lean's default decreasing tactic, or else an `efficient def`, whose termination suffix is copied to the cost function. The loop variant, stated once.
- **A recursor** (`Nat.rec`, `casesOn`) is priced like a `match`, one per decision. An induction hypothesis becomes the companion of the recursive result, so a recursion that returns a function charges each application of the result.

```
efficient def gcd' (a b : Nat) : Nat :=
  if h : b = 0 then a else gcd' b (a % b)
termination_by b
decreasing_by exact Nat.mod_lt _ (Nat.pos_of_ne_zero h)
#eval gcd'.cost 12 18                                  -- 13
```

The subtle cases, 3: classes and the elaborated term

- **Classes.** The arithmetic classes have registered unit costs, whatever the carrier. Another method is unfolded through a concrete instance, or gets a companion on an abstract one.
- **Casts** along equalities are free.
- **A `let`-bound function** is inlined, so its calls are priced at the function it stands for.
- **An honest `opaque`** is priced through its value; a `partial def` is refused by name.
- **A function stored in a structure** that is not a parameter (a global constant, or one built on the spot) is reached by unfolding the structure to its constructor.

The experiment

In the Ironwood formalization and its CompElliptic dependency, we have 241 non-test `assert_computable` pins. We make a seeded split, half of each file: **121 development, 120 evaluation**.

1. Rewrite the development pins as `assert_efficient`; make them pass, changing the machinery as needed; record the ones that genuinely are not efficient.
2. Freeze the machinery.
3. Rewrite the evaluation pins; count: *just works*, *needs effort*, *cannot pass*.

The development pins are the training set, the evaluation pins the held-out test set. The question is how much of a real formalization a derivation written against half of it covers.

Results, 1: development

Ironwood's 112 development pins, one build per round, eleven rounds:

round	1	3	4	5	7	11
passing	7	11	33	39	48	**50**
bought by	—	companions	prelude, casts, `casesOn`	field companions	eta, `opaque`, `Nat.rec`	instance companions, `rec` with function results

What the rounds removed, in order: higher-order code everywhere (lambdas to `map`, continuations, oracles); adversaries and interfaces as structures of functions; class methods on abstract instances (`+` on a field, `default`, `Fintype.elems`); the elaborator's leftovers (`casesOn`, `Decidable.rec`, `Nat.rec`, casts); `opaque` barriers that hide a computable value.

Frozen after round 11 at **50 of 112**. Still failing (62): functions bound by lambdas or patterns (26); quotients (11); methods on instance variables (8); functions in fields of non-parameter structures (7); partial applications (4); cost functions that do not apply (4); timeouts (2).

Results, 2: evaluation

The held-out pins, rewritten as `assert_efficient` at the frozen machinery, one build:

	just works	fails at the frozen machinery
Ironwood (110)	**41**	69
CompElliptic (10)	**7**	3
total (120)	**48**	72

The 69 Ironwood failures: functions bound by lambdas or patterns (39); `Int.natAbs` (8); fields of non-parameter structures (6); methods on instance variables (5); partial applications (4); quotients (4); timeouts (3). CompElliptic's three: quotients.

No kind is new: each already appears among the development leftovers. Registered costs for the three named primitives rescue none; each caller then stops at the next unsupported shape.

Development: 50 of 112 after eleven rounds of fitting. Evaluation: 48 of 120 at the first build; CompElliptic's nine development pins, never fitted against, all pass.

Gaps

- **Not priced yet.** Quotients: `Finset` and `Multiset` go through `Quot.lift`, and the cost depends on the representative that the quotient hides. Functions bound by a pattern or by a lambda inside data (continuations, functions inside sum types): no companion reaches them.
- **Needs help.** A well-founded recursion with a hand-written measure: an `efficient def` with the termination suffix, or a registered cost. `partial def`. Two large definitions time out.
- **The cost model is coarse.** One unit per arithmetic operation, whatever the carrier, as in `time_fun_0`; one `Nat` resource, with no split between group and field operations.
- **No bound is proved automatically.** The derived cost function is the whole output.
- **Trust.** The derivation is a meta-program, not a kernel check. What it guarantees is that `f.cost` is defined by the rules above from `f`'s elaborated body.

Conclusions

- The cost of a reduction can be **derived**, not annotated: a few hundred lines on top of Lean's equation-lemma records and pre-definition machinery.
- The adversary's cost is a **companion** argument, so the result is symbolic in the adversary, which is the shape that a concrete-security theorem needs.
- The derivation fails loudly at a named primitive, a missing termination argument, or an unsupported shape, which is exactly where human judgement belongs.
- Next: bounds proved automatically from structural and well-founded measures; costs in several resources (group operations); a standalone package.

Code and questions

`Zcash/Meta/Efficient.lean` on the branch `meta/efficient-keyword` of the Ironwood repository:

https://github.com/zcash/ironwood/pull/228

